

UNIT III NOTES

Monte Carlo prediction and control algorithms, state and action exploration, and MCTS

UNIT OVERVIEW

Monte Carlo (MC) methods are a class of **model-free Reinforcement Learning algorithms** that learn directly from **complete episodes** of experience. Unlike Dynamic Programming (DP), Monte Carlo methods do **not require knowledge of the environment model** (transition probabilities and rewards). They estimate value functions by averaging the returns observed from multiple episodes.

This unit covers:

- Monte Carlo Methods
- Monte Carlo Prediction
- Monte Carlo Control
- Exploring Starts
- ϵ -Greedy Exploration
- State and Action Exploration
- Monte Carlo Tree Search (MCTS)

1. Introduction to Monte Carlo Methods

Definition

Monte Carlo methods are learning algorithms that estimate value functions by averaging the returns obtained from complete episodes.

Unlike Dynamic Programming,

- No transition probability is required.
- Learning occurs only after an episode terminates.
- Suitable for episodic tasks.

Characteristics

- Model-free learning
 - Learns from experience
 - Requires complete episodes
 - Estimates expected returns by averaging
 - Applicable to stochastic environments
-

Comparison with Dynamic Programming

Dynamic Programming	Monte Carlo
Requires environment model	No model required
Uses Bellman equations	Uses sample episodes
Updates after every state	Updates after complete episode
Exact computation	Approximate estimation
Suitable for known environments	Suitable for unknown environments

Important Terminologies

Episode

An episode is a complete sequence of interaction.

Example:

Start

↓

State A

↓

State B

↓

State C

↓

Goal

Episode Ends

Return (G)

The cumulative discounted reward received from time step t onwards.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where

- γ = Discount factor
 - G = Return
-

State Value Function

Expected return from state s.

$$V(s) = E[G_t \mid S = s]$$

Action Value Function

Expected return after taking action a .

$$Q(s, a) = E[G_t' \mid S = s, A = a]$$

2. Monte Carlo Prediction

Definition

Monte Carlo Prediction estimates the value function of a **given policy** by averaging returns observed in multiple episodes.

The policy remains fixed.

Only the state values are estimated.

Objective

Estimate

$$V^\pi(s)$$

for every state.

Basic Idea

1. Generate many episodes using policy π .
2. Record returns from each visited state.
3. Compute average return.
4. Update state values.

Mathematical Formula

If state s is visited n times,

$$V(s) = \frac{G_1 + G_2 + \dots + G_n}{n}$$

where

- G = observed return
-

First-Visit Monte Carlo Prediction

Definition

Only the **first occurrence** of a state in an episode is considered.

Repeated visits within the same episode are ignored.

Algorithm

Step 1

Initialize

$$V(s) = 0$$

Return(s)=empty

Step 2

Generate an episode.

Step 3

For every state

If first occurrence

Compute return.

Step 4

Append return.

Step 5

Average returns.

Pseudocode

Initialize $V(s)$

Repeat

Generate Episode

For each state

if first visit

 Compute Return G

 Append G

$V(s) = \text{Average}(\text{Returns})$

Until enough episodes

Example

Episode

$A \rightarrow B \rightarrow C \rightarrow \text{Goal}$

Rewards

5

3

10

Return from A

$$5 + 3 + 10 = 18$$

Return from B

$$3 + 10 = 13$$

Return from C

10

Every-Visit Monte Carlo Prediction

Definition

Every occurrence of a state is considered.

Repeated visits are included.

Difference

First Visit	Every Visit
Uses first occurrence	Uses every occurrence
Less computation	More computation

First Visit

Lower variance

Standard method

Every Visit

Slightly higher computation

Faster use of available samples

Monte Carlo Prediction Flowchart

Initialize Value Function

|

Generate Episode

|

Compute Returns

|

Average Returns

|

Update State Values

|

More Episodes?

Yes —► Repeat

No

Estimated Value Function

Advantages

- Simple implementation
 - No environment model required
 - Converges to true values with enough episodes
-

Limitations

- Must wait until episode ends
 - High variance
 - Slow convergence
-

3. Monte Carlo Control

Definition

Monte Carlo Control learns the **optimal policy** directly from experience.

Unlike prediction,

Prediction

↓

Policy Fixed

Control

↓

Policy Improved

Objective

Find

$$\pi^*$$

Process

Generate Episodes

↓

Estimate Action Values

↓

Improve Policy

↓

Repeat

Action Value Update

$$Q(s, a) = \text{Average}(\text{Returns})$$

Policy Improvement

$$\pi(s) = \arg \max_a Q(s, a)$$

Monte Carlo Control Algorithm

Step 1

Initialize random policy.

Step 2

Generate episode.

Step 3

Estimate action values.

Step 4

Improve policy greedily.

Step 5

Repeat.

Pseudocode

Initialize Policy π

Initialize $Q(s,a)$

Repeat

Generate Episode

Estimate Returns

Update Q

Improve Policy

Until Policy Stable

Control Flow

Random Policy

|

Generate Episode

|

Estimate $Q(s,a)$

|

Improve Policy

|

Policy Stable?

No —▶ Repeat

Yes

Optimal Policy

4. Exploring Starts (ES)

Definition

Exploring Starts is a strategy that ensures **every state-action pair has a non-zero probability of being selected as the starting point of an episode.**

This guarantees that all actions are explored sufficiently.

Why Exploring Starts?

If an action is never tried, the agent cannot know whether it is better than the current choice.

Example

State S

Actions

- Left
- Right
- Up
- Down

Exploring Starts randomly begins episodes with each action, ensuring that all actions are sampled over time.

Advantages

- Guarantees complete exploration.
- Helps Monte Carlo Control converge to the optimal policy.

Disadvantages

- Difficult to implement in real-world systems because we usually cannot choose arbitrary starting states and actions.

5. State and Action Exploration

Definition

Exploration means trying different states and actions to discover better rewards rather than always choosing the currently best-known action.

Two competing objectives are:

- **Exploration:** Gather new information.
- **Exploitation:** Use existing knowledge to maximize reward.

A good RL agent balances both.

Why Exploration is Needed

Suppose a robot always turns **right** because it has received good rewards in the past. It may never discover that turning **left** leads to a much larger reward. Exploration prevents the agent from getting stuck with a suboptimal policy.

Exploration vs Exploitation

Exploration

Tries new actions

Gains information

May receive lower short-term reward

Improves long-term learning

Exploitation

Chooses the best-known action

Gains immediate reward

Maximizes current reward

May miss better actions

ϵ -Greedy Exploration

Definition

The ϵ -greedy policy selects:

- The best action with probability **$1 - \epsilon$**
- A random action with probability **ϵ**

where $0 \leq \epsilon \leq 1$.

Formula

$$\pi(a | s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|}, & \text{if } a = \arg \max_a Q(s, a) \\ \frac{\epsilon}{|A(s)|}, & \text{otherwise} \end{cases}$$

Example

Assume:

- $\epsilon = 0.1$
- Four possible actions

Then:

- Best action: 90% chance (plus its share of the random choice)
 - Random exploration: 10% distributed across all actions
-

Advantages

- Simple and widely used.
 - Prevents the agent from getting stuck in local optima.
 - Guarantees continual exploration.
-

Limitations

- Random exploration may still choose poor actions.
- Choosing ϵ is important:
 - Too large $\rightarrow$ excessive exploration.
 - Too small $\rightarrow$ insufficient exploration.

Other Exploration Strategies

Strategy	Description
Exploring Starts	Random starting state-action pair
ϵ -Greedy	Mostly greedy with occasional random actions
Soft Policy	Assigns non-zero probability to all actions
Optimistic Initialization	Starts with high value estimates to encourage exploration
Upper Confidence Bound (UCB)	Chooses actions using reward estimates and uncertainty

6. Monte Carlo Tree Search (MCTS)

Definition

Monte Carlo Tree Search (MCTS) is a heuristic search algorithm that combines **tree search** with **Monte Carlo simulations** to determine the most promising action.

Instead of evaluating every possible future state, MCTS explores only the most promising branches of the search tree.

Applications

- Chess
- Go (AlphaGo)
- Robotics
- Autonomous planning
- Video games

- Route planning
-

Main Idea

Repeat many simulations.

Each simulation consists of four stages.

Selection

↓

Expansion

↓

Simulation

↓

Backpropagation

Phase 1: Selection

Starting from the root node, repeatedly choose the child with the highest selection score until a leaf node is reached.

A common rule is the **Upper Confidence Bound for Trees (UCT)**:

$$UCT = \bar{X}_j + c \sqrt{\frac{\ln N}{n_j}}$$

where:

- $\bar{X}_j$: average reward of child j
- N : number of visits to the parent
- n_j : number of visits to child j
- C : exploration constant

The first term favors exploitation; the second favors exploration.

Phase 2: Expansion

If the selected node is not terminal and has unexplored actions:

- Create one or more child nodes.
 - Add them to the search tree.
-

Phase 3: Simulation (Rollout)

From the new node, simulate actions (often randomly or using a simple policy) until a terminal state is reached.

The outcome of the simulation (win, loss, score, etc.) is recorded.

Phase 4: Backpropagation

Propagate the simulation result back to the root.

For each node on the path:

- Increase visit count.
- Update average reward/value estimate.

This information improves future selections.

MCTS Algorithm

Step 1

Initialize the search tree with the current state as the root.

Step 2

Repeat until the computation budget (time or number of simulations) is exhausted:

1. Selection
2. Expansion
3. Simulation
4. Backpropagation

Step 3

Choose the action corresponding to the child of the root with the highest visit count (or highest value).

MCTS Flowchart

Current State (Root)

|

Selection

|

Expansion

|

Simulation (Rollout)

|

Backpropagation

|

More Simulations?

Yes —► Repeat

No

Best Action

Example: Tic-Tac-Toe

Suppose the AI must decide its next move.

1. Start from the current board (root).
 2. Select the most promising move.
 3. Expand a new board position.
 4. Play random moves until the game ends.
 5. Record whether the AI wins or loses.
 6. Update all nodes visited.
 7. Repeat thousands of times.
 8. Select the move that has the highest visit count or average reward.
-

Advantages of MCTS

- Does not require an exact environment model.
- Focuses computation on promising branches.
- Can handle very large search spaces.
- Produces increasingly better decisions with more simulations.
- Widely used in modern game-playing AI.

Limitations of MCTS

- Computationally expensive.
- Performance depends on the quality and number of simulations.
- May be slow for problems requiring immediate decisions.
- Rollout policy quality affects performance.

Comparison of Monte Carlo Methods

Feature	MC Prediction	MC Control	MCTS
Purpose	Estimate state values	Learn optimal policy	Search for the best action
Learns Policy?	No	Yes	Yes (through search)
Requires Model?	No	No	Typically uses a simulator or generative model for rollouts
Uses Episodes?	Yes	Yes	Yes (simulated rollouts)
Output	State value function	Optimal policy and action values	Best action from current state

Comparison: Dynamic Programming vs Monte Carlo vs MCTS

Feature	Dynamic Programming	Monte Carlo	MCTS
Environment Model	Required	Not required	Simulator/generative model typically required
Uses Bellman Equations	Yes	No	No
Learns from Experience	No	Yes	Yes (through simulations)
Episode Required	No	Yes	Simulated episodes
Suitable for Unknown Environments	No	Yes	Often, if a simulator exists
Main Use	Exact planning	Model-free learning	Decision-time planning

Key Examination Points

- **Monte Carlo Prediction** estimates the value function of a fixed policy by averaging returns from complete episodes.
- **First-Visit MC** updates a state's value using only its first occurrence in an episode, while **Every-Visit MC** uses all occurrences.
- **Monte Carlo Control** estimates action values and repeatedly improves the policy using greedy or ϵ -greedy selection.
- **Exploring Starts** ensures every state-action pair has a chance to be explored.
- **ϵ -Greedy Exploration** balances exploration and exploitation by choosing the greedy action most of the time while occasionally selecting a random action.
- **Monte Carlo Tree Search (MCTS)** consists of four phases: **Selection** → **Expansion** → **Simulation (Rollout)** → **Backpropagation**.
- **UCT** is commonly used in MCTS to balance exploration and exploitation during node selection.

- **Monte Carlo methods** require complete episodes and do not need a transition model, making them well suited for model-free reinforcement learning in episodic environments.